

Tema 4

Evaluación de Sistemas Informáticos. Monitorización

Objetivos

Entender el concepto de monitor de actividad de un Sistema Informático y sus diferentes utilidades e implementaciones.

Conocer las características fundamentales de un monitor a nivel de sistema operativo y a nivel de aplicación concreta (profilers).

Comprender el papel que desempeñan los monitores para evaluar el rendimiento de un Sistema Informático ante una carga real.

Saber interpretar adecuadamente la información que aporta un monitor.

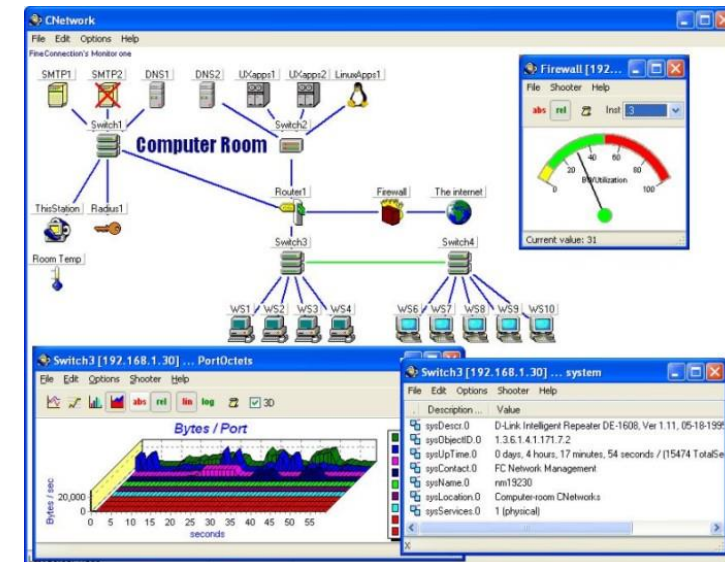
Plantear trabajo de la asignatura

Contenidos

Concepto de monitor de actividad.

Monitorización a nivel de sistema.

Monitorización a nivel de aplicación.



Bibliografía



Evaluación y modelado del rendimiento de los sistemas informáticos. Xavier Molero, C. Juiz, M. Rodeño. Pearson Educación, 2004.

Capítulo 2



The art of computer system performance analysis. R. Jain. John Wiley & Sons, 1991.

Capítulos 7 y 8



System performance tuning. G.D. Musumeci, M. Loukides. O'Reilly, 2002.

Capítulos 1 y 2



Linux performance and tuning guidelines. E.Ciliendo, T.Kunimasa. IBM Redpaper, 2007.

Capítulos 1 y 2



Linux man pages. <http://www.linuxmanpages.com/>.

Monitores de actividad

Conceptos básicos

Como decíamos hace ya mucho, mucho tiempo...

“Un Sistema no es bueno ni malo *per se*, sino que se adapta mejor o peor a un tipo determinado de carga”

Carga (*load*): conjunto de tareas que ha de hacer un sistema

- Programas, datos y órdenes de los usuarios

Carga de prueba (*test workload*)

- Carga empleada en un estudio de evaluación

Variables que reflejan la carga

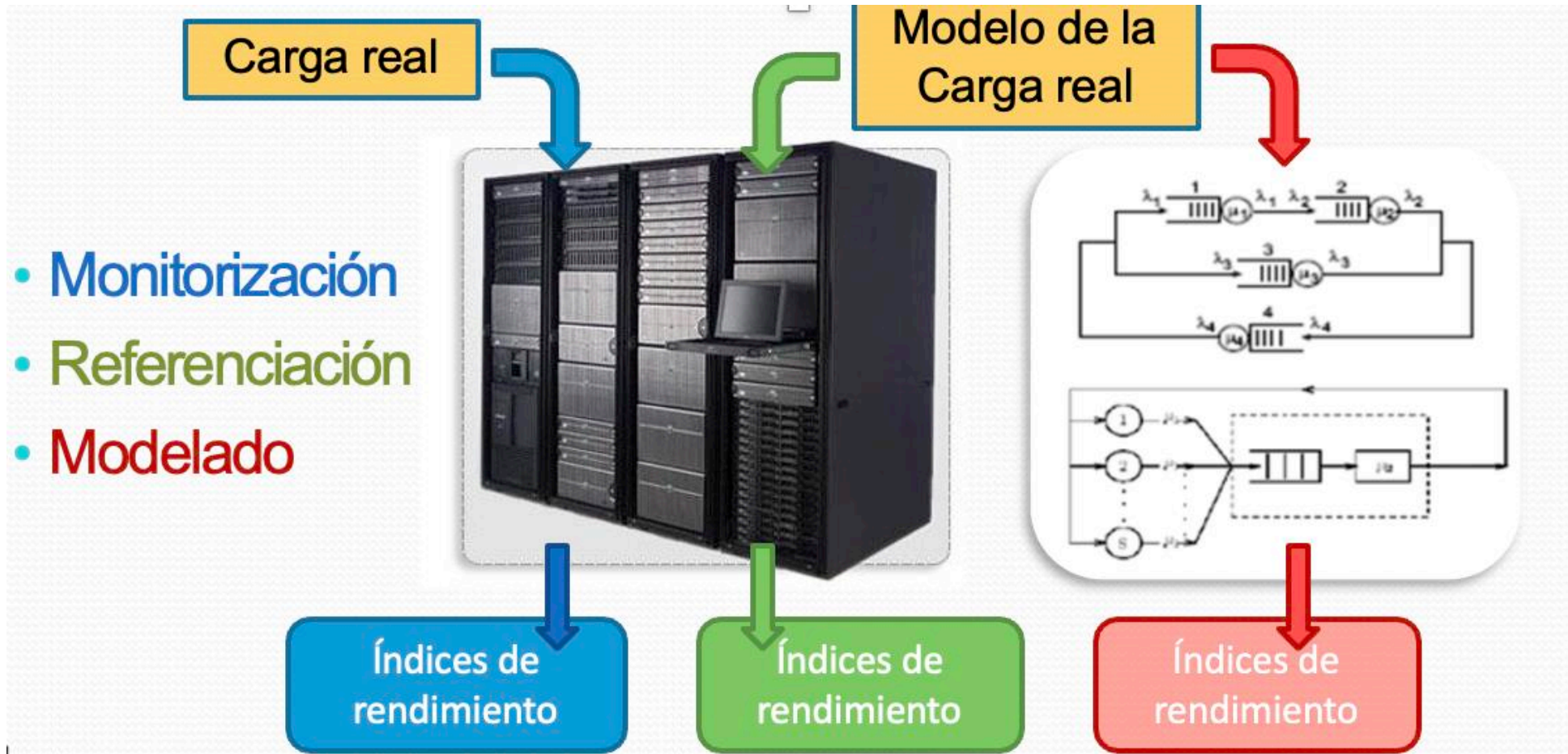
- Número de programas simultáneos en ejecución
- Accesos por unidad de tiempo a un servidor de páginas web
- Peticiones por unidad de tiempo a una base de datos
- Etc.

Cómo podemos **evaluar**? (medir, comparar o predecir el rendimiento)



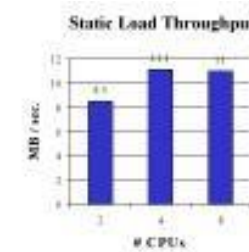
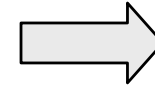
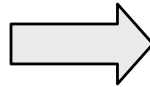
- **Monitorización** (medir)
 - Herramientas de medida sobre el sistema real
- **Referenciación** (*benchmarking*) (comparar)
 - Comparación del rendimiento de sistemas modelados o reales
- **Modelado** (predecir)
 - Reproducción del comportamiento del sistema
 - Métodos analíticos (redes de colas, cadenas de Markov, redes de Petri, ...)
 - Simulación discreta (CSIM, SMPL, Simula, ...)

¿Cómo podemos **evaluar**? (medir, predecir o comparar el rendimiento)



¿Cómo se mide la actividad de un sistema?

Carga



Medidas cuantitativas

Magnitudes medibles (tiempos de respuesta, utilizaciones, productividades...)

Procesadores: Utilización, número de procesos, interrupciones, cambios de contexto, etc.

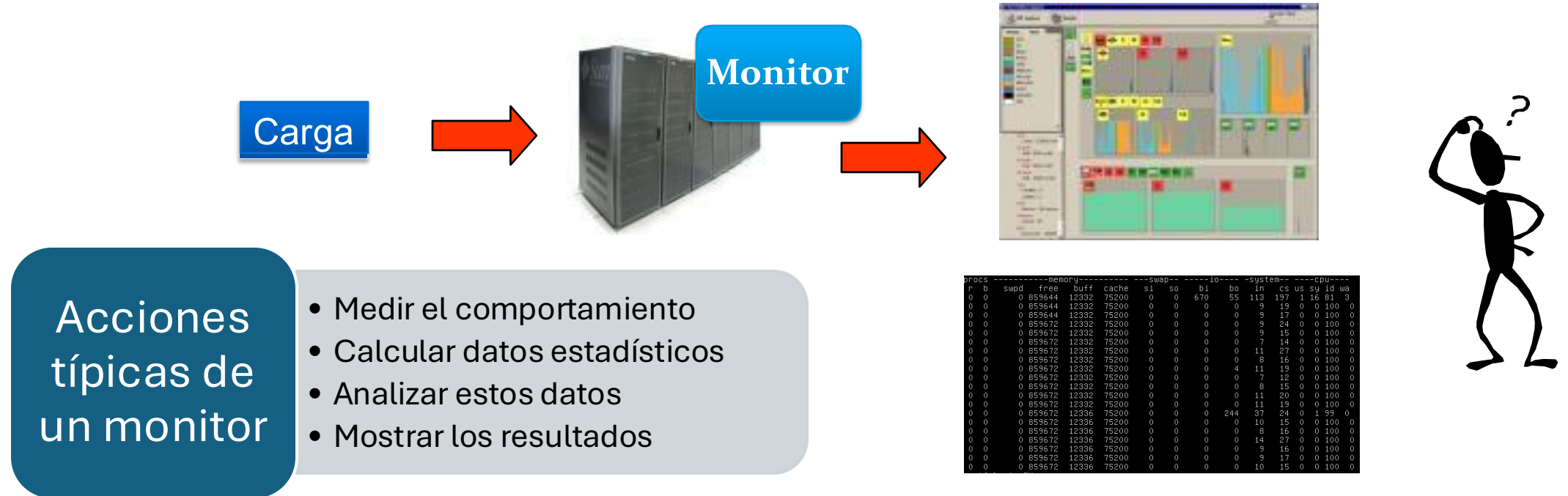
Memoria: memoria física utilizada, fallos de caché, fallos de página, frecuencia de uso de memoria de intercambio, fallos de TLB, etc.

Discos: lecturas/escrituras por unidad de tiempo, longitud de las colas de espera, tiempo de espera medio por acceso, etc.

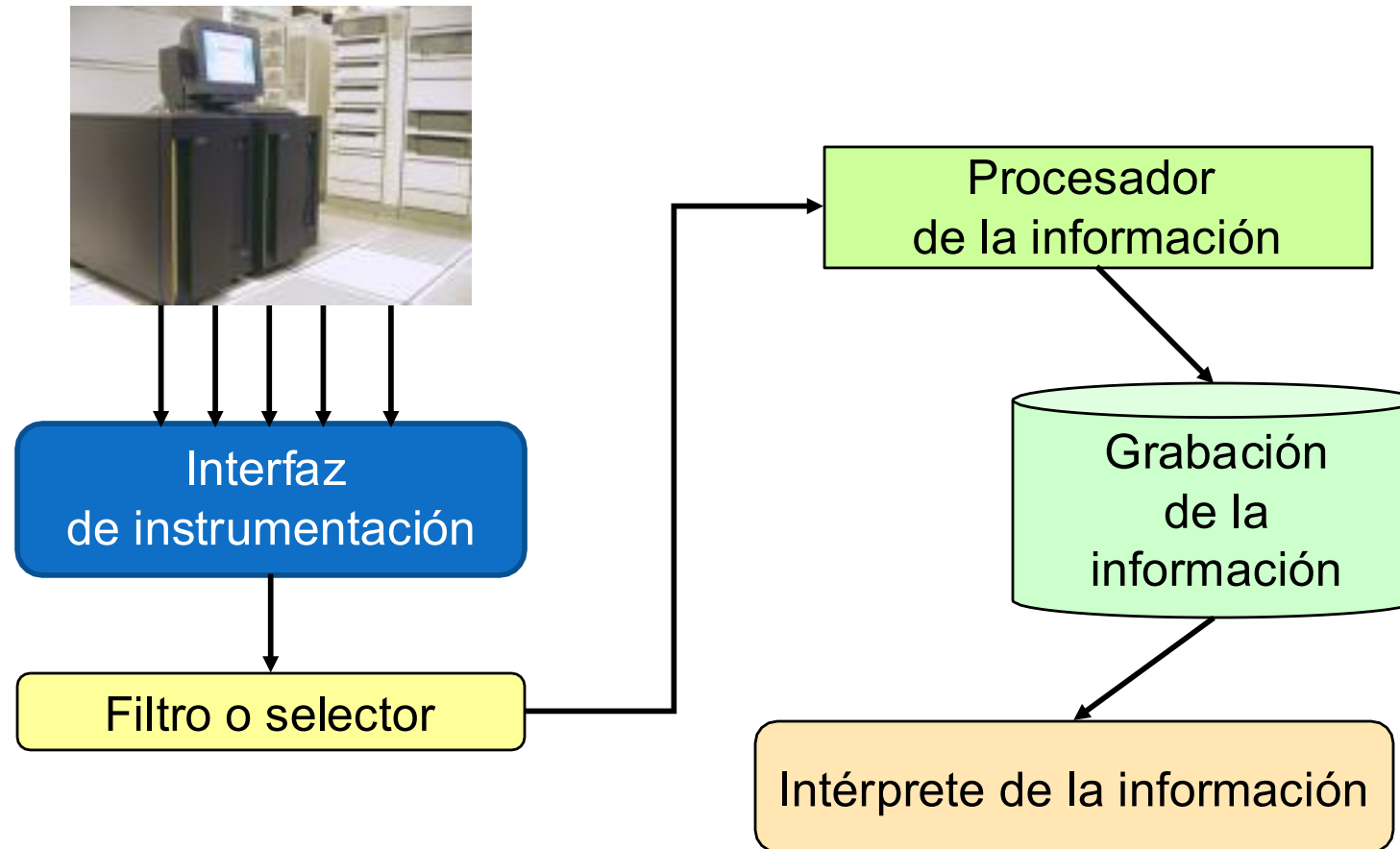
Red: paquetes recibidos/enviados, colisiones por segundo, paquetes perdidos, etc.

Definición de monitor de actividad

Herramienta diseñada para observar/analizar la actividad de un sistema informático mientras es utilizado (carga real)



Esquema conceptual de un monitor





Utilidad de los monitores de actividad

Administrador

- Conocer la utilización de los recursos (detección de cuellos de botella)
- Ajustar los parámetros del sistema (sintonización)

Programador

- Conocer las partes más críticas o de mayor frecuencia de uso de una aplicación.

Analista/Ingeniero

- Parametrizar la carga real y calcular los parámetros de entrada a modelos del sistema (analíticos o simulación)
- Predecir cargas futuras (capacity planning)
- Conocer qué hardware hay que reconfigurar / añadir

Sistema

- Adaptarse dinámicamente a la carga

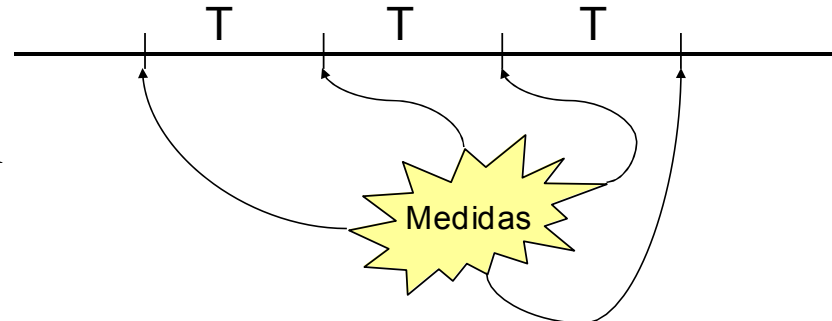
Tipos de monitores: ¿cuándo se mide?

Cada vez que ocurre un evento (*event-driven monitor*)

- Evento: Cambio en el estado del sistema
- Volumen de información recogida: Depende de la frecuencia de los eventos
- Ejemplos de eventos:
 - Inicio/fin de la ejecución de un programa.
 - Acierto/fallo en memoria cache.
 - Atención a un dispositivo periférico.
 - Abrir/cerrar un fichero, etc.

A intervalos regulares de tiempo (*sampling monitor*)

- Análisis estadístico de datos más fácil
- El volumen de información global recogida depende del periodo de muestreo (T)



Tipos de monitores: ¿cómo se mide?

Software

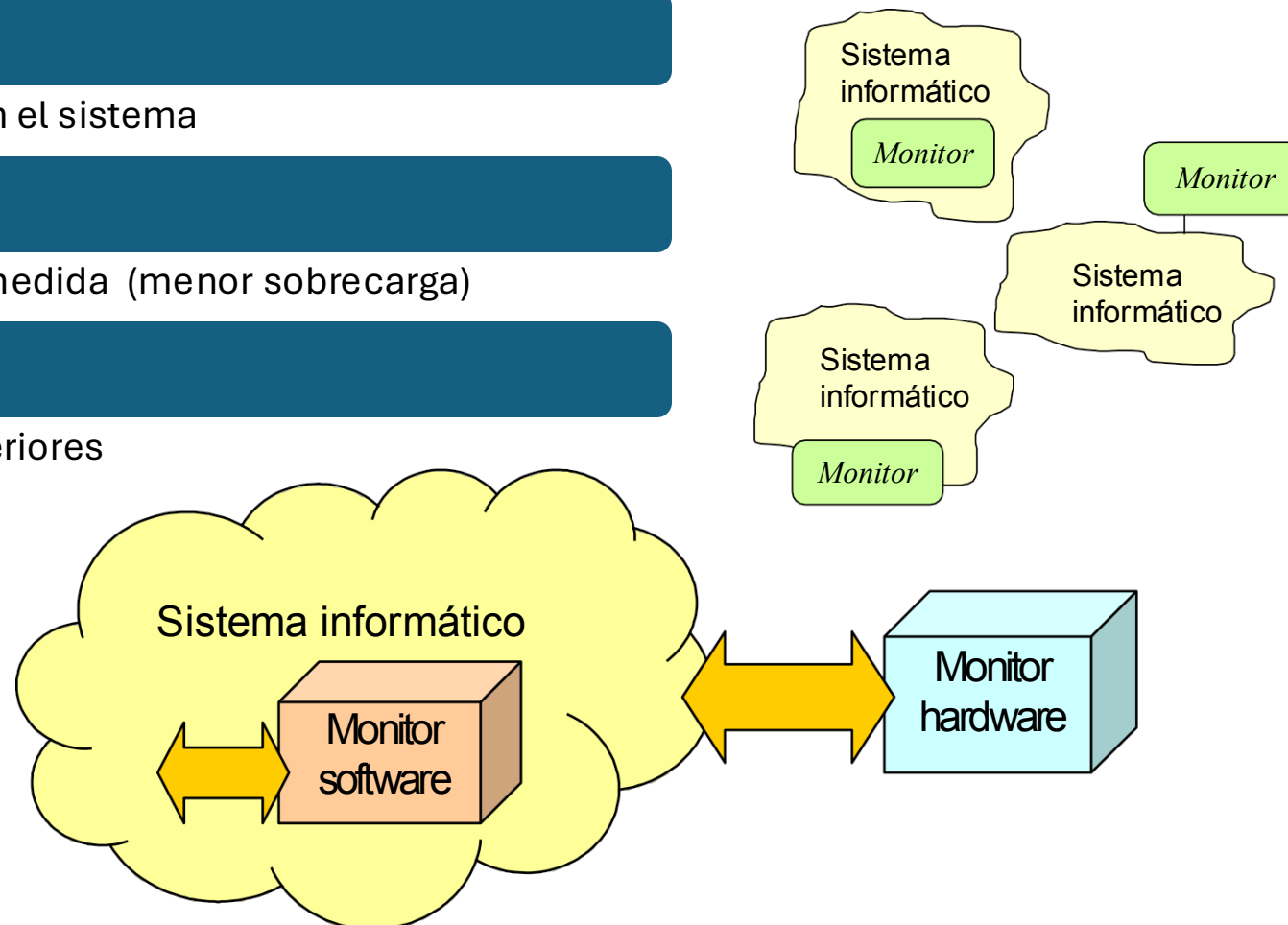
- Programas instalados en el sistema

Hardware

- Dispositivos físicos de medida (menor sobrecarga)

Híbridos

- Utiliza los dos tipos anteriores



Tipos de monitores: ¿existe interacción con el analista/usuario/administrador?

No existe, la consulta sobre los resultados se realiza aparte mediante otra herramienta independiente al proceso de monitorización: monitores tipo batch (*batch monitors*).

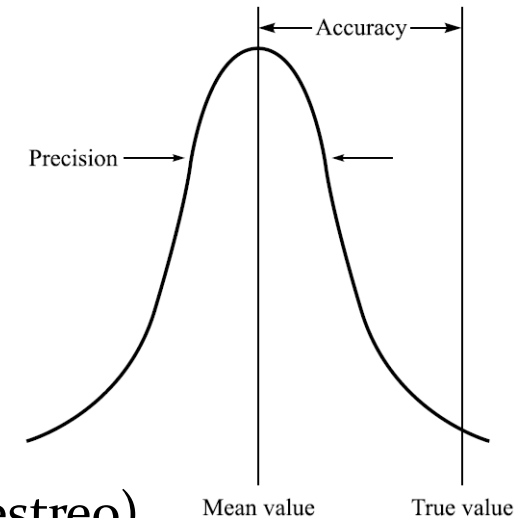
Sí existe, durante el propio proceso de monitorización: monitores interactivos (*on-line monitors*).

- El propio usuario puede modificar, en tiempo real, las variables monitorizadas, el periodo de muestreo, el tipo de eventos a monitorizar, realizar representaciones gráficas, etc.



Atributos que caracterizan a un monitor

- **Exactitud** de la medida (*Accuracy*): ¿Cómo se aleja el valor medido del valor real?
- **Precisión** (*Precision*): ¿Cuál es la dispersión de la medida?
- **Resolución** del monitor: ¿Cuánto tiene que cambiar el valor a medir para detectar un cambio?
- **Frecuencia de Muestreo** (*Sampling Time*):
 - ¿Cada cuánto tiempo tomamos la medida? (monitores basados en muestreo)
- **Tasa de Entrada** (*Input Rate*): ¿Cuál es la frecuencia máxima de ocurrencia de los eventos que el monitor puede observar? (monitores por eventos)
- **Anchura** de entrada (*Input Width*): ¿Cuántos bits de información se almacenan por cada medida que toma el monitor?
- **Sobrecarga** (*Overhead*): ¿cuánto tiempo de cómputo le roba el monitor al sistema? El instrumento de medida puede perturbar el funcionamiento del sistema.



Cálculo de la sobrecarga en un monitor

La ejecución de las instrucciones del monitor se lleva a cabo utilizando recursos del sistema monitorizado

$$\text{Sobrecarga}_{\text{RECURSO}} = \frac{\text{Uso del recurso por parte del monitor}}{\text{Capacidad total del recurso}}$$

Ejemplo de cálculo

El monitor se activa cada 5 s y cada activación del mismo usa el procesador durante 6 ms

$$\text{Sobrecarga}_{\text{CPU}} = \frac{6 \times 10^{-3} \text{ s}}{5 \text{ s}} = 0.0012 = 0.12\%$$

Monitorización a nivel de sistema

El directorio `/proc` (Unix)

Es una carpeta en RAM utilizada por el núcleo de Unix como interfaz con las estructuras de datos del kernel.

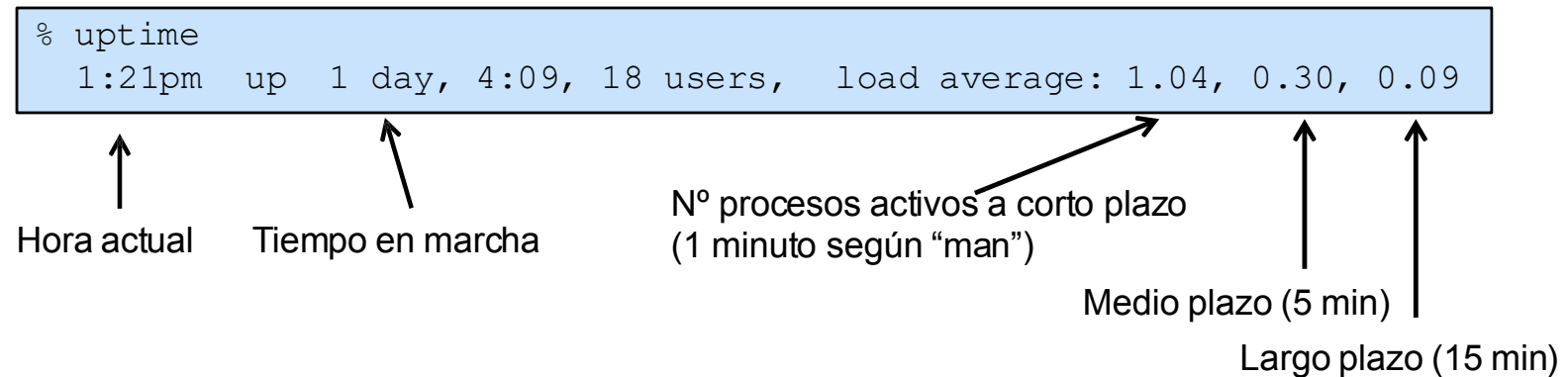
A través de `/proc` podemos:

- Acceder a información global sobre el S.O. : `loadavg`, `uptime`, `cpuinfo`, `meminfo`, `mounts`, `net`, `ide`, `kmsg`, `cmdline`, `slabinfo`, `filesystems`, `diskstats`, `devices`, `interrupts`, `stat`, `swap`, `version`, `vmstat` ...
- Acceder a la información de cada uno de los procesos del sistema. (`/proc/[pid]`): `stat`, `status`, `statm`, `mem`, `smaps`, `cmdline`, `cwd`, `environ`, `exe`, `fd`, `task`...
- Acceder y, a veces, modificar algunos parámetros del kernel del S.O. (`/proc/sys`): `dentry_state`, `dir-notify-enable`, `dquot-max`, `dquot-nr`, `file-max`, `file-nr`, `inode-max`, `inode-nr`, `lease-break-time`, `mqueue`, `super-max`, `super-nr`, `acct`, `domainname`, `hostname`, `panic`, `pid_max`, `version`, `net`, `vm`...

La mayoría de los monitores de Linux usan como fuente de información este directorio.

uptime

Tiempo que lleva el sistema en marcha y la carga media que soporta



Estimación aproximada del nivel de carga (aunque depende de las prestaciones esperadas de cada sistema)

- Operación normal: hasta 3
- Muy alta: entre 4 y 7
- Excesivamente alta: mayor que 10

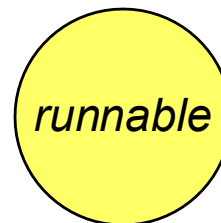
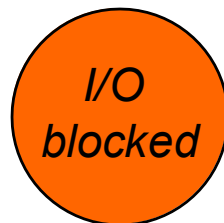
Carga del sistema Unix

Estados básicos de un proceso

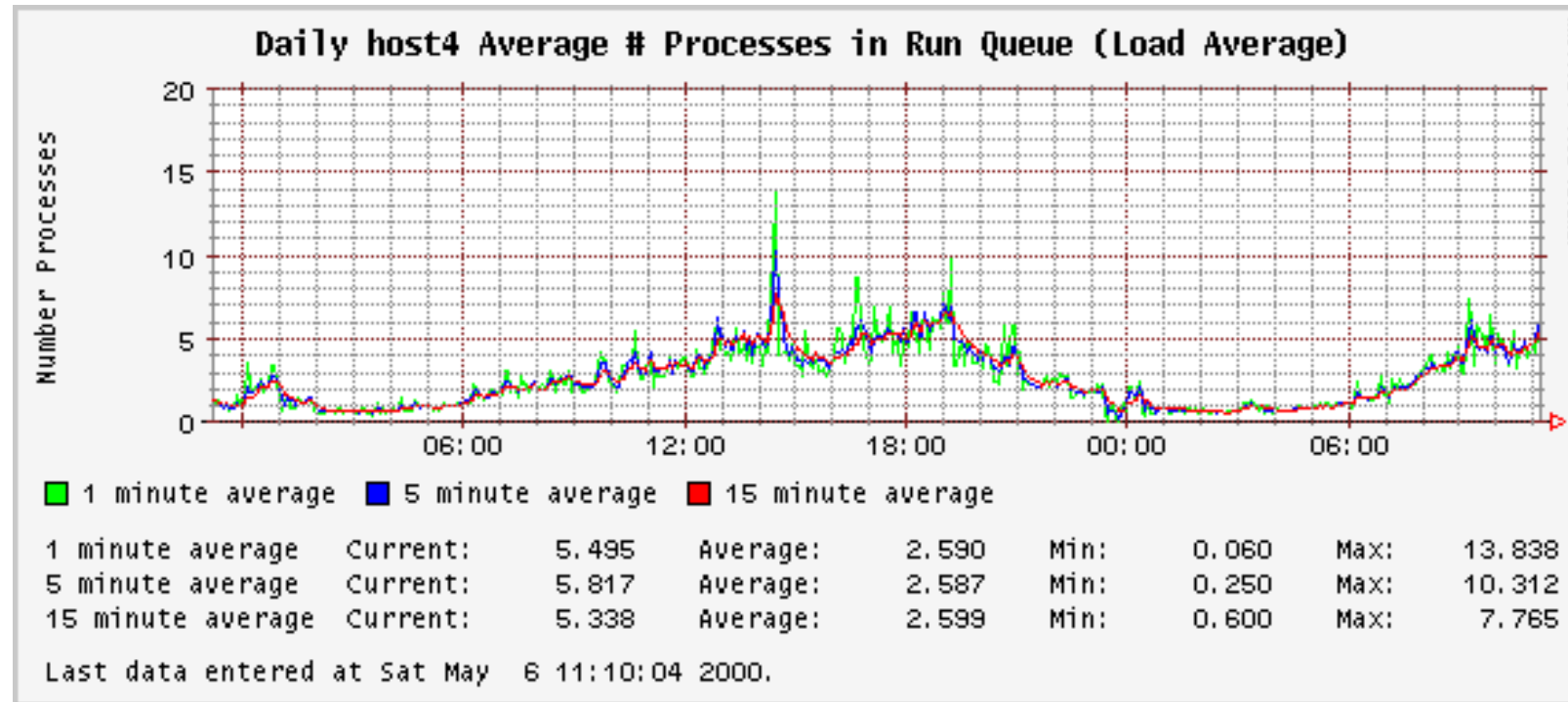
- En ejecución (*running*) o en la cola de ejecución (*runnable*).
- Durmiendo esperando a que se complete una operación de E/S para continuar (*uninterruptible sleep* = *I/O blocked*).
- Bloqueado esperando a un evento del usuario o similar (p.ej. una pulsación de tecla) (*interruptible sleep*).

La cola de procesos del núcleo (*run queue*) está formada por aquellos que pueden ejecutarse

Carga del sistema (*system load*): número de procesos en modo *running*, *runnable* o *I/O blocked*.



Evolución típica de la carga media



©José Manuel Soto Hidalgo

Evaluación de Sistemas Informáticos. Monitorización

ps (*process status*)

Información sobre el estado actual de los procesos del sistema

Es una de las herramientas más importantes empleadas en tareas de monitorización

```
$ ps aur
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
sotillo   29951 55.9   0.1 1448    384 pts/0    R    09:16  0:11 tetris
carlos    29968 50.6   0.1 1448    384 pts/0    R    09:32  0:05 tetris
javier    30023  0.0   0.5 2464   1492 pts/0    R    09:27  0:00 ps aur
```

Tiene una gran cantidad de parámetros (aquí se presentan algunas)

```
***** simple selection ***** selection by list *****
-A, -e all processes                      -C by command name
-N negate selection                      -G by real group ID
-a all w/ tty except session leaders     -U by real user ID
-d all except session leaders            -g by session
-p by process ID                        t  by tty
T  all processes on this terminal        -s proc. in sessions given
a  all w/ tty, including other users     -t by tty
-u by effective user ID                  r  only running processes
U  processes for specified users         x  processes w/o controlling ttys
```


Información aportada por `ps`

USER

Usuario que lanzó el proceso

%CPU, %MEM

Porcentaje de procesador y memoria física usada

SIZE (o VSIZE)

Memoria (KiB) virtual total asignada al proceso

RSS (*resident size*)

Memoria (KiB) física ocupada por el proceso

STAT

R (*running or runnable*), D (*I/O blocked*), S (*interruptible sleep*), T

(*stopped*), Z (*zombie: terminated but not died*)

N (*lower priority = running niced*), < (*higher priority = not nice*)

s (*session leader*), + (*in the foreground process group*), W (*swapped*)

top

Muestra cada T segundos : carga media, procesos, consumo de memoria...

Normalmente se ejecuta en modo interactivo (se puede cambiar T, las columnas seleccionadas, la forma de ordenar las filas, etc.)

```
8:48am  up 70 days, 21:36,    1 user, load average: 0.28, 0.06, 0.02
47 processes: 44 sleeping, 3 running, 0 zombie, 0 stopped
CPU states: 99.6% user,    0.3% system,    0.0% nice,
              0.0% idle
Mem:      256464K av,    234008K used,    22456K free, 0K shrd, 13784K
buff Swap:          136512K av,              4356K
used, 132156K free    5240K cached

  PID USER      PR  NI  SIZE  RSS  SHARE  STAT  LC  %CPU  %MEM  TIME  COMMAND
    I
9826 carlos    0   0   388   388   308  R      0  99.6   0.1  0:22  simulador
9831 sotill  19   0   976   976   776  R      0   0.3   0.3  0:00  top
    o
   1 root    20   0    76    64    44  S      0   0.0   0.0  0:03  init
   2 root    20   0     0     0     0  SW     0   0.0   0.0  0:00  keventd
   4 root    20  19     0     0     0  SWN    0   0.0   0.0  0:00  ksoftiq
   5 root    20   0     0     0     0  SW     0   0.0   0.0  0:13  kswapd
   6 root     2   0     0     0     0  SW     0   0.0   0.0  0:00  bdf flush
   7 root    20   0     0     0     0  SW     0   0.0   0.0  0:10  kdated
   8 root    20   0     0     0     0  SW     0   0.0   0.0  0:01  kinoded
  11 root     0 -20     0     0     0  SW<    0   0.0   0.0  0:00  recoved
```

vmstat (*virtual memory statistics*)

Paging (paginación), *swapping*, interrupciones, cpu

% vmstat 1 6															
procs				memory				swap		io		system			cpu
r	b	w	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id
0	0	0	868	8964	60140	342748	0	0	23	7	222	199	1	4	95
0	0	0	868	8964	60140	342748	0	0	0	14	283	278	0	7	93
0	0	0	868	8964	60140	342748	0	0	0	0	218	212	6	2	93
0	0	0	868	8964	60140	342748	0	0	0	0	175	166	3	3	94
0	0	0	868	8964	60140	342752	0	0	0	2	182	196	0	7	93
0	0	0	868	8968	60140	342748	0	0	0	18	168	175	3	8	89

Procesos: *r* (*runnable*), *b* (*I/O blocked*), *w* (*swapped out*)

Bloques por segundo transmitidos: *bi* (*blocks in*), (*blocks out*)

KB/s entre memoria y disco: *si* (*swapped in*), *so* (*swapped out*)

in (*interrupts por second*), *cs* (*context switches*)

Con otros argumentos, puede dar información sobre acceso a discos, estado de ciertos contadores de eventos y otras estadísticas de memoria.

who y w

who: quién está conectado al sistema (*logged on*)

```
sotillo      :0      Oct 30 15:07 (console)
javier       pts/0    Oct 30 17:45 (uco.es)
```

w: quién está conectado al sistema (*logged on*) y qué hace

```
% w
  1:38p   up  4:27, 18 users,  load average: 0.04, 0.03, 0.04
m
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
sotillo   ttyt      pepino.uco.es    9:17am   2:02m   2:48   0.48s  -sh
1
fede      ttyt                        10:28am   51:02   0.14s   0.03s  rlogin ma
2
javi      :0        decsai.ugr.es    1:20pm   ?        7:32   ?      -
pperez    ttyt                        10:02am   29:22   0.18s   0.14s  ssh tiberio.
3
```

Información sobre los discos

df (*filesystem disk space usage*)

```
$ df
```

Filesystem	1k-blocks	Used	Available	Use%	Mounted on
/dev/hda2	9606112	3017324	6100816	34%	/
/dev/hdb1	12775180	9236405	3140445	75%	/home

du (*file space usage*)

```
$ du doc
```

```
160      doc/cartas
432      doc
```

hdparm (*hard disk parameters*)

```
$ hdparm -g /dev/hda
```

```
/dev/hda:
```

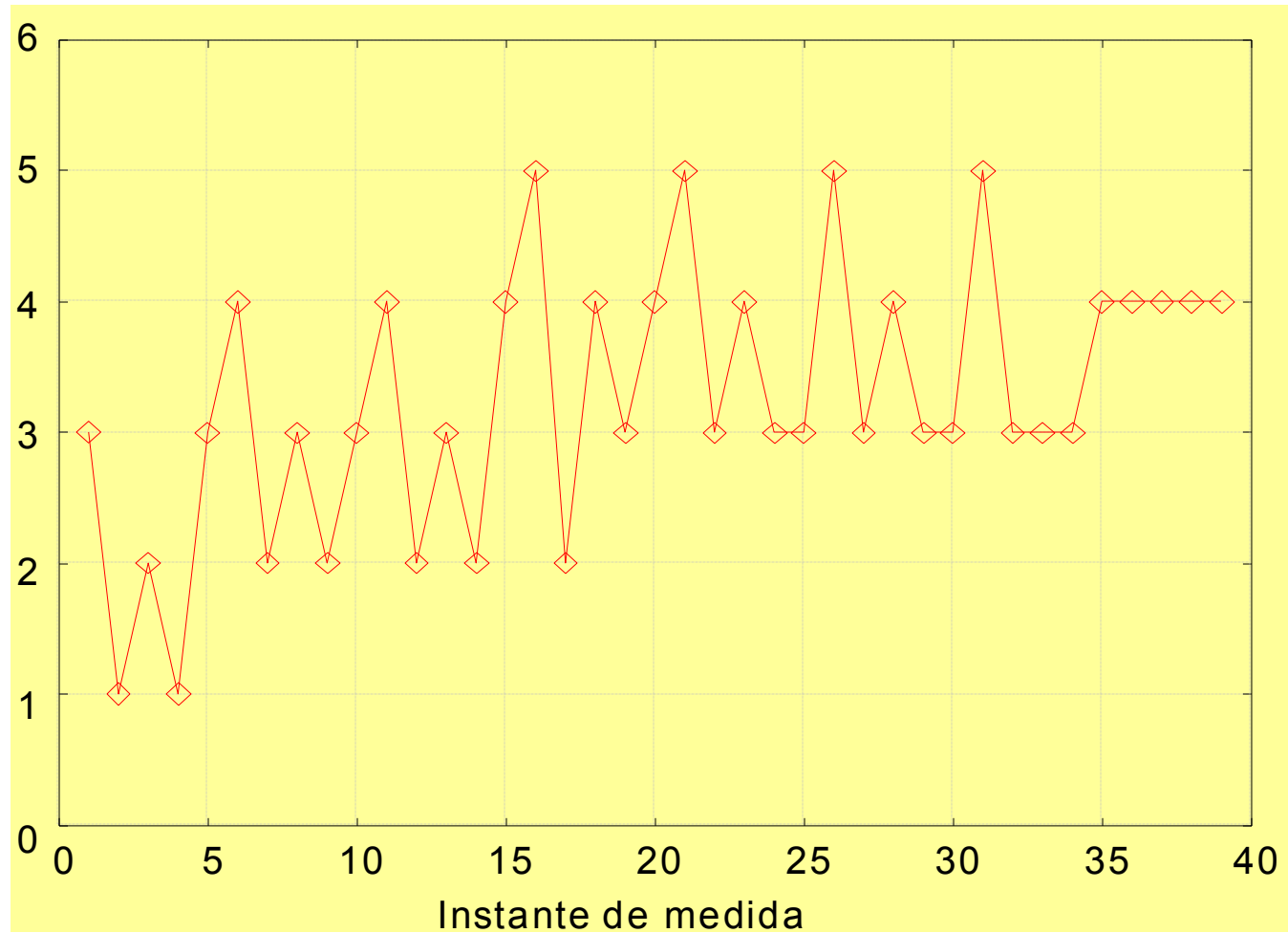
```
geometry      = 790/255/63, sectors = 12706470, start = 0
```

```
$ hdparm -tT /dev/hda
```

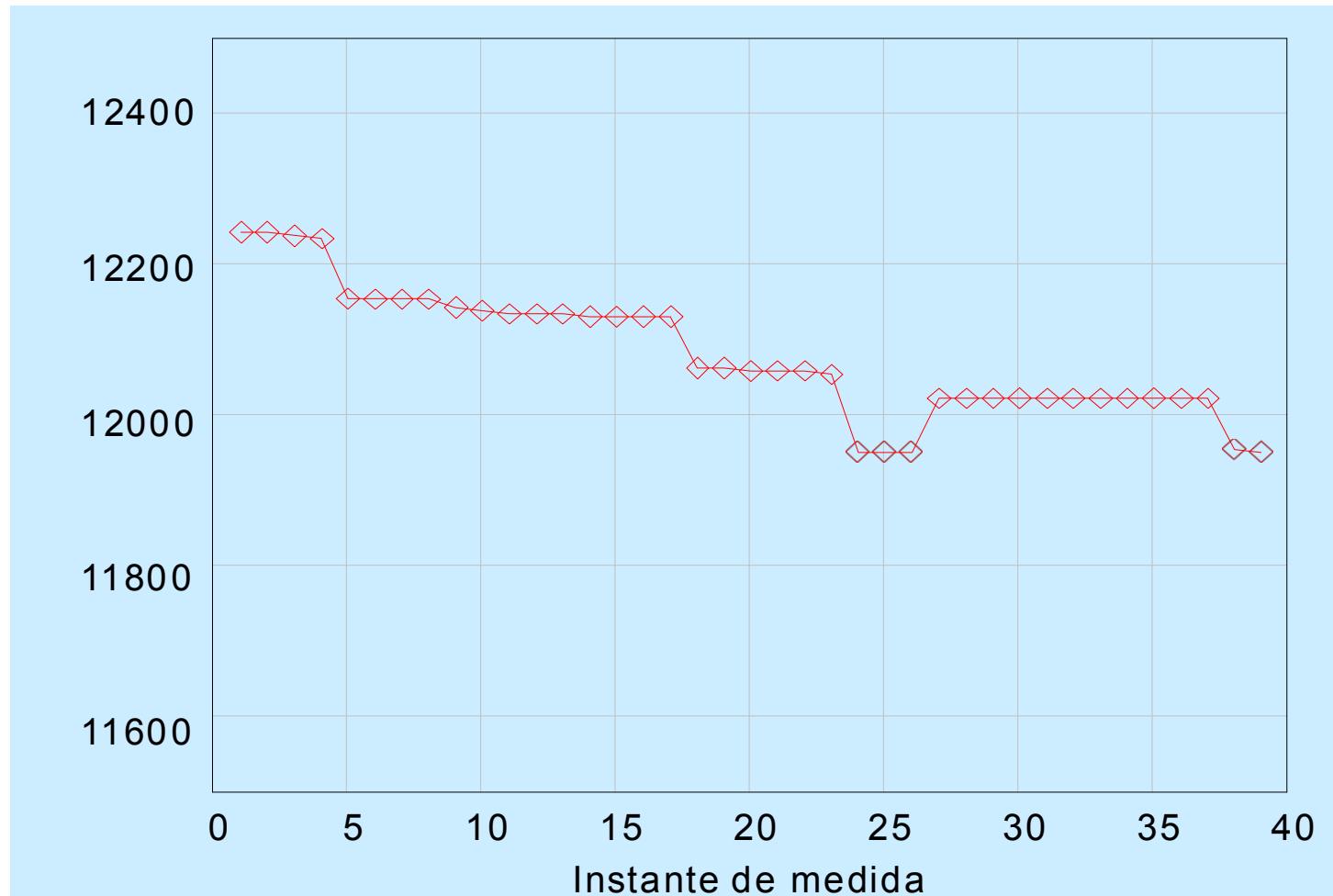
```
Timing buffer-cache          128 MB in   1.15 seconds =111.30 MB/sec
reads:
```

```
Timing buffered disk reads:  64 MB in   6.04 seconds = 10.60 MB/sec
```

Procesos activos



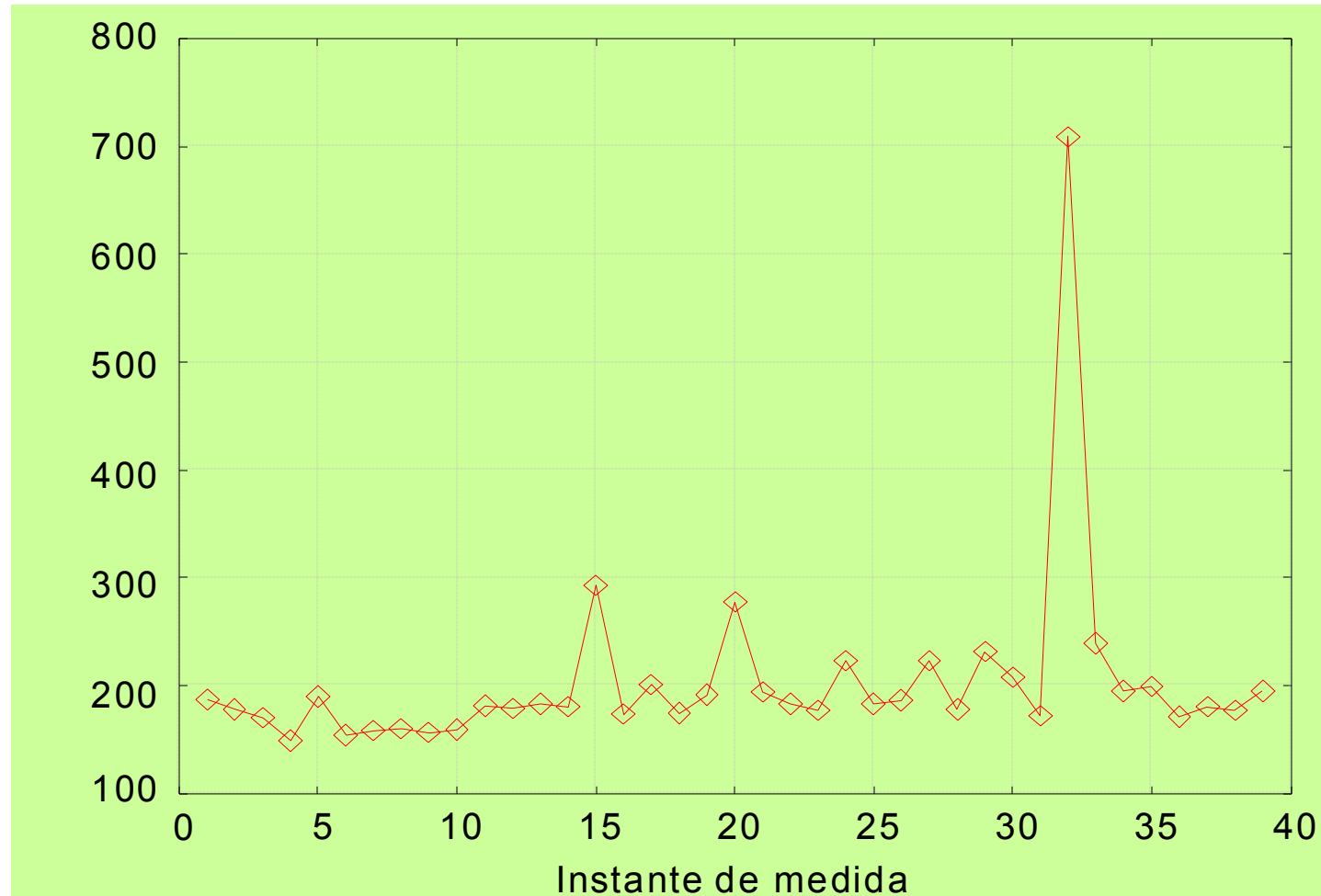
Capacidad de memoria libre



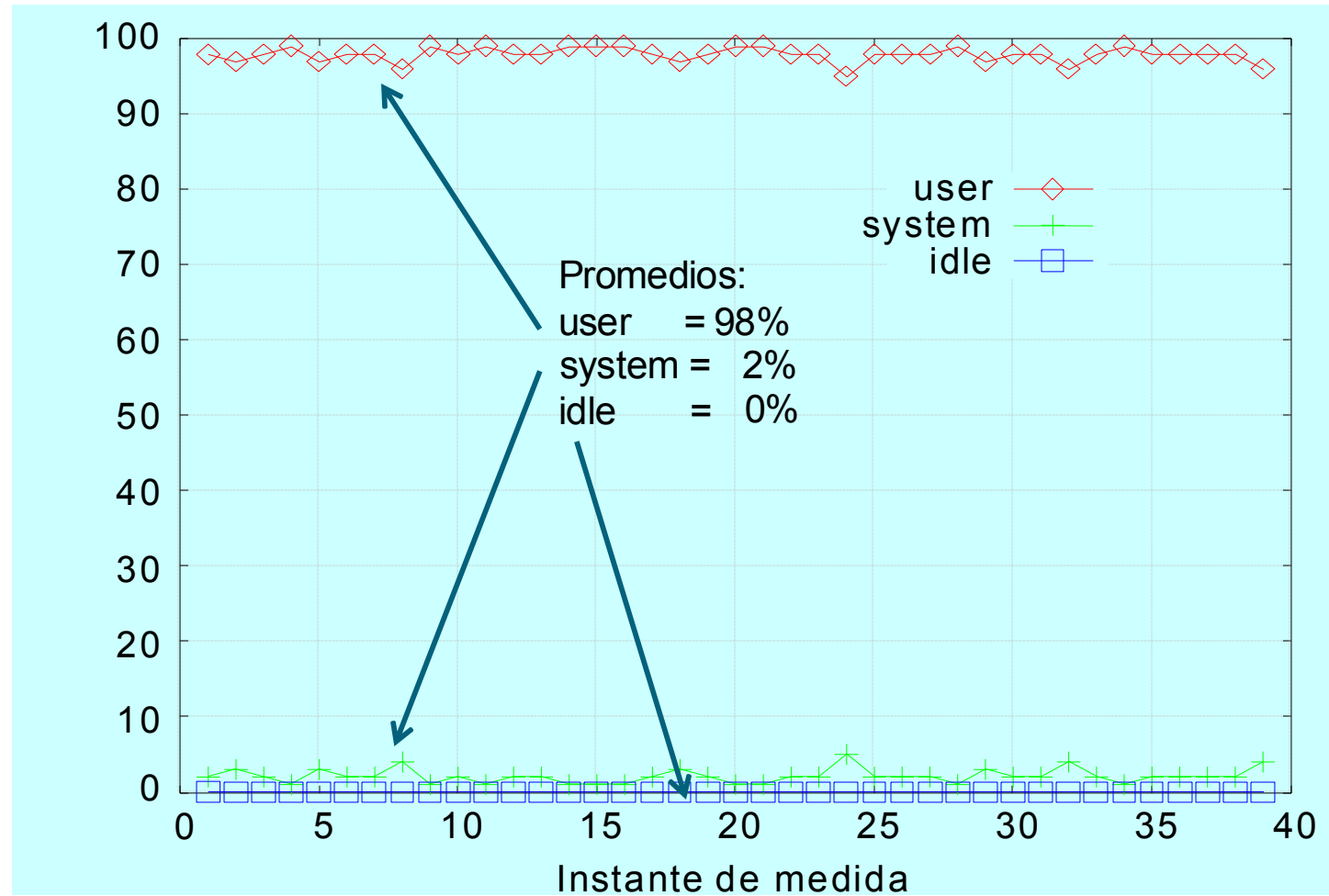
Tema 4

Interrupciones por segundo

©José Manuel Soto Hidalgo



Utilización del procesador



El paquete de monitorización: SysStat



El monitor `sar`

`sar` (*system activity reporter*)

Muy utilizado por los administradores de sistemas Unix en la detección de cuellos de botella (*bottlenecks*)

Información sobre todo el sistema

Actual: qué está pasando el día de hoy, o ahora mismo, al sistema

Histórica: qué ha pasado en el sistema en otros días pasados

Ficheros históricos

`saDD`, donde los dígitos `DD` indican el día del mes

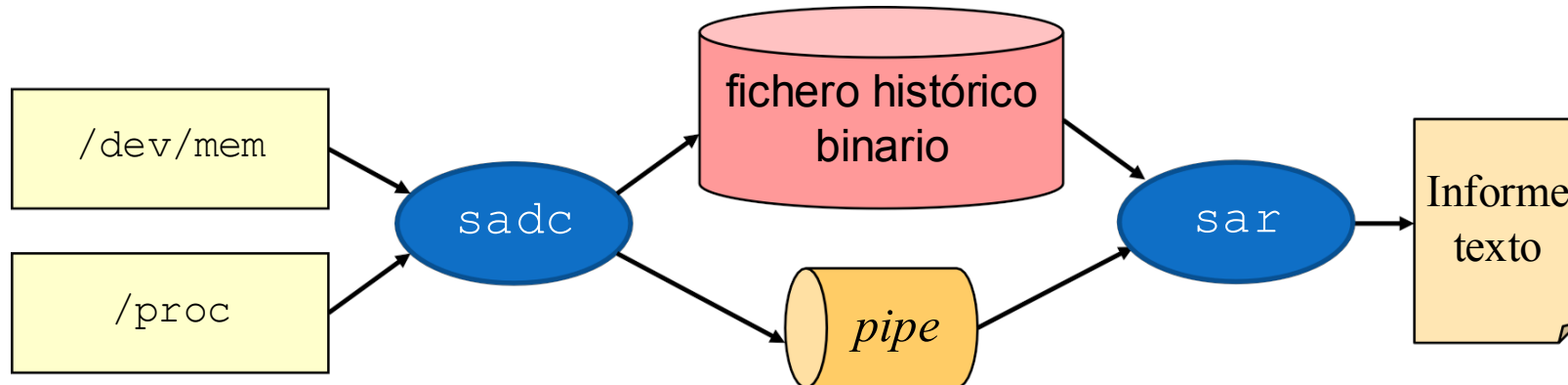
Hace uso de contadores estadísticos del núcleo del sistema operativo ubicados en los directorios `/proc` y `/dev/mem`

Disponibilidad en internet y ya incluido en muchas distribuciones de Linux

<http://sebastien.godard.pagesperso-orange.fr/>

Funcionamiento del monitor `sar`

- Se basa en dos órdenes complementarias
 - `sadc` (*system-accounting data collector*)
 - Recoge los datos estadísticos (lectura de contadores) y construye un registro en formato binario (*back-end*)
 - `sar`
 - Lee los datos binarios que recoge `sadc` y los traduce a un formato legible por nosotros en formato texto (*front-end*)



Parámetros de sar

Gran cantidad de parámetros (puede funcionar en modo batch o en modo interactivo)

- A Toda la información disponible
- b Estadísticas de transferencias de E/S
- B Paginación de la memoria virtual
- d Transferencias para cada disco
- e Hora de fin de la monitorización
- f Fichero de donde extraer la información
- I Estadísticas sobre interrupciones
- n Conexión de red
- o Guardar salida en un fichero
- P Mostrar estadísticas por cada procesador
- q Tamaño de la cola y carga media del sistema
- r Utilización de memoria
- R Estadísticas sobre la memoria
- s Hora de comienzo de la monitorización
- S Estadísticas sobre utilización de espacio swap
- u Utilización del procesador
- w Cambios de contexto
- W Estadísticas sobre *swapping*

Ejemplos de ejecución de `sar`

Ejecución interactiva

```
sar 2 30
```

Información recogida sobre el día de hoy

```
sar
```

```
sar -d -s 10:00 -e 12:00
```

```
sar -A
```

```
sar -u
```

Información recogida en otro día anterior

```
sar -f /var/log/sa/sa02
```

```
sar-P -f /var/log/sa/sa06
```

```
sar -d -s 10:00 -e 12:00 -f /var/log/sa/sa04
```

Tema 4

Ejemplo de salidas del monitor sar

Utilización de los procesadores (sistema biprocesador)

```
$ sar
00:00:00      CPU      %user      %nice      %system      %idle
00:05:00      all       0.09       0.00       0.08       99.83
00:10:00      all       0.01       0.00       0.01       99.98
...
11:15:00      all       0.02       0.00       0.02       99.96
11:20:00      all       0.44       0.00       0.20       99.36
11:25:00      all       0.05       0.00       0.02       99.92
```

Actividad del sistema de entrada/salida

```
$ sar -b
00:00:00      tps      rtps      wtps      bread/s      bwrtn/s
00:05:00      0.74      0.39      0.35        7.96        3.27
00:10:01      0.09      0.00      0.09        0.00        0.91
00:15:00      0.15      0.00      0.14        0.03        1.36
00:20:00     65.12     59.96      5.16     631.62     162.64
```

Los datos sobre la actividad

Se utiliza un fichero histórico de datos por cada día

Se programa la ejecución de `sadc` un número de veces al día con la utilidad “cron” del sistema Unix

Por ejemplo, una vez cada 5 minutos

Cada ejecución de `sadc` añade un registro binario con los datos recogidos al fichero histórico del día

```
%ls /var/log/sa
-rw-r--r--  1 root  root    3049952 Sep 30 23:55 sa30
-rw-r--r--  1 root  root    3049952 Oct  1 23:55 sa01
-rw-r--r--  1 root  root    3049952 Oct  2 23:55 sa02
-rw-r--r--  1 root  root    3049952 Oct  3 23:55 sa03
-rw-r--r--  1 root  root    3049952 Oct  4 23:55 sa04
-rw-r--r--  1 root  root    3049952 Oct  5 23:55 sa05
-rw-r--r--  1 root  root    3049952 Oct  6 23:55 sa06
-rw-r--r--  1 root  root    3049952 Oct  7 23:55 sa07
-rw-r--r--  1 root  root    2372320 Oct  8 18:45 sa08
```

Día
actual



Análisis de un ficheros histórico

Ejemplo

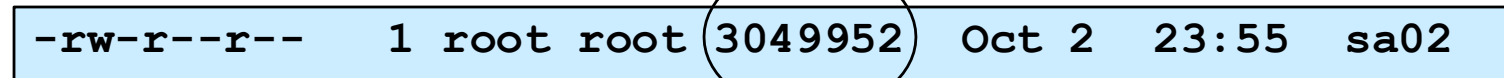
El fichero histórico de un día ocupa 3.049.952 bytes (unos 3 MB)

La orden `sadc` se ejecuta cada 5 minutos

Cada hora se recogen 12 muestras

Al día se recogen $24 \times 12 = 288$ muestras

Por tanto, cada registro ocupa 10.3 KB



```
-rw-r--r-- 1 root root 3049952 Oct 2 23:55 sa02
```

Fichero `sa02`
(día 2 de octubre)

Otras herramientas de Sysstat

`mpstat` (*processors related statistics*)

```
$ mpstat -P 1 3 5
12:07:03      CPU   %user   %nice %system   %idle   intr/s
12:07:06        1 100.00    0.00    0.00    0.00    63.00
12:07:09        1 100.00    0.00    0.00    0.00    66.00
12:07:12        1 100.00    0.00    0.00    0.00    44.00
12:07:15        1 100.00    0.00    0.00    0.00    74.00
12:07:18        1 100.00    0.00    0.00    0.00    50.00
Average:        1 100.00    0.00    0.00    0.00    59.40
```

`iostat` (*input/output statistics*)

```
$
iostat      %user   %nice    %sys    %idle
cpu-
avg:
          3.70    0.02    0.48    95.81
Device:      tps    Blq_read/s  Blq_wrtn/s  Blq_read  Blq_wrtn
dev2-0        0.00         0.00         0.00       133         0
dev3-0        0.55         4.53         6.62   11726226   17108122
dev3-1        0.01         0.00         0.61     2698    1590072
```

Programa SarCheck

Herramienta para

- Análisis de prestaciones

- Sintonización, planificación de la capacidad

Sistemas Sun Solaris, HP-UX, AIX y Linux x86

Basado en el monitor `sar`

Utiliza `gnuplot` para generar gráficos

Genera informes en formato HTML

- Sección de recomendaciones

- Sección de análisis de recursos

- Sumario de estadísticas, etc.

www.sarcheck.com

SarCheck™

*UNIX Performance Tuning Simplified..
and Linux Performance Tuning too!*

Tema 4

SarCheck[®] UNIX Performance Tuning Simplified...
and Linux Performance Tuning tool!

APTITUNE CORPORATION

Kernel activity reported in English

Easily understand resource utilization

Text or HTML output with graphs

Kernel tuning recommendations

Capacity planning



SarCheck is a Linux & UNIX performance analysis and performance tuning tool. It is designed to help you with performance management on most Oracle Solaris, AIX, Linux, and HP-UX systems by making recommendations and explaining them with plain text, supporting graphs, and tables.

Evaluation Copy Sample Reports SarCheck Video Platforms Customer Reviews About Contact

- Solaris
- AIX
- Linux
- HP-UX

Learn more about SarCheck...

1
Play the
SarCheck Video

2
View a Sample
SarCheck Report

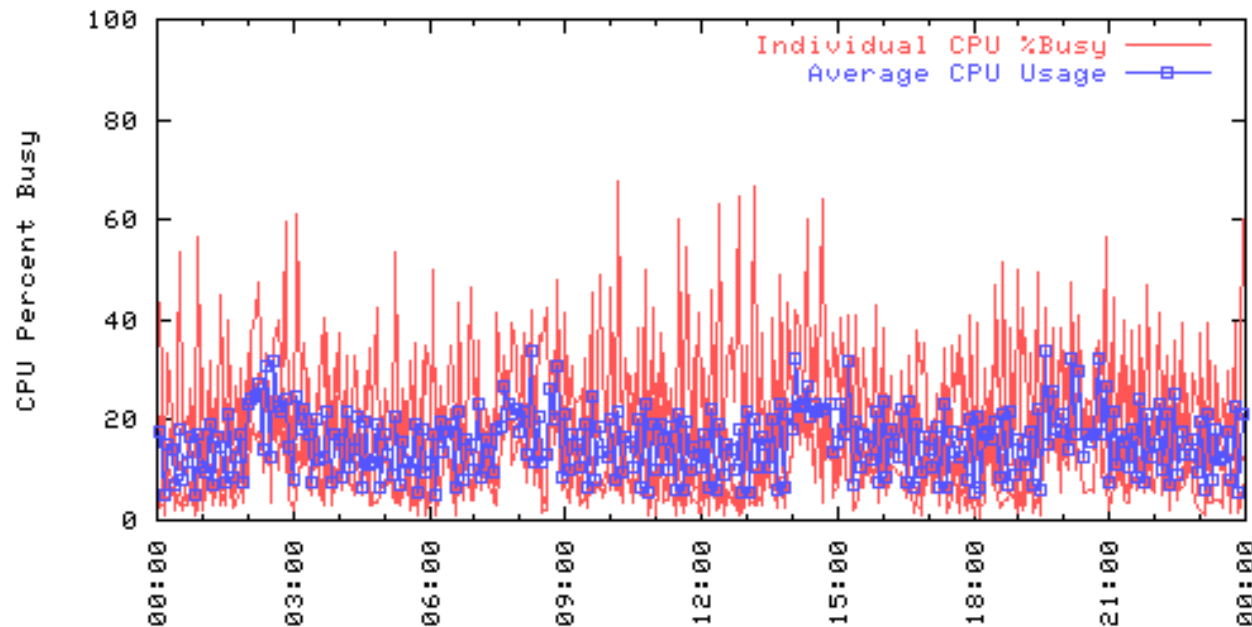
3
Try a FREE
Evaluation Copy

©José Manuel Soto Hidalgo

Tema 4

Ejemplo informe generado por SarCheck

- Average CPU utilization was only 15.7 percent. This indicates that spare capacity exists within the CPU. If any performance problems were seen during the monitoring period, they were not caused by a lack of CPU power. CPU utilization peaked at 34.00 percent from 08:10:01 to 08:15:01. A CPU upgrade is not recommended because the current CPU had significant unused capacity.



©José Manuel Soto Hidalgo

Monitorización a nivel de aplicación

Profilers

Objetivo

Observar el comportamiento de una aplicación concreta con el fin de obtener información para poder optimizar su código.

Información que pueden proporcionar las herramientas de análisis (*profilers*)

¿Cuánto tiempo tarda en ejecutarse el programa?
¿Qué parte de ese tiempo es de usuario y cuál de sistema? ¿Cuánto tiempo se pierde por las operaciones de E/S?

¿En qué parte del código pasa la mayor parte de su tiempo de ejecución?

¿Cuántas veces se ejecuta cada línea de programa?

¿Cuántas veces se llama a un procedimiento y desde dónde?

¿Qué funciones se llaman desde un determinado procedimiento?

¿Cuántos fallos de caché/TLB/página genera el programa?

Monitorización a nivel de aplicación (*profiling*)



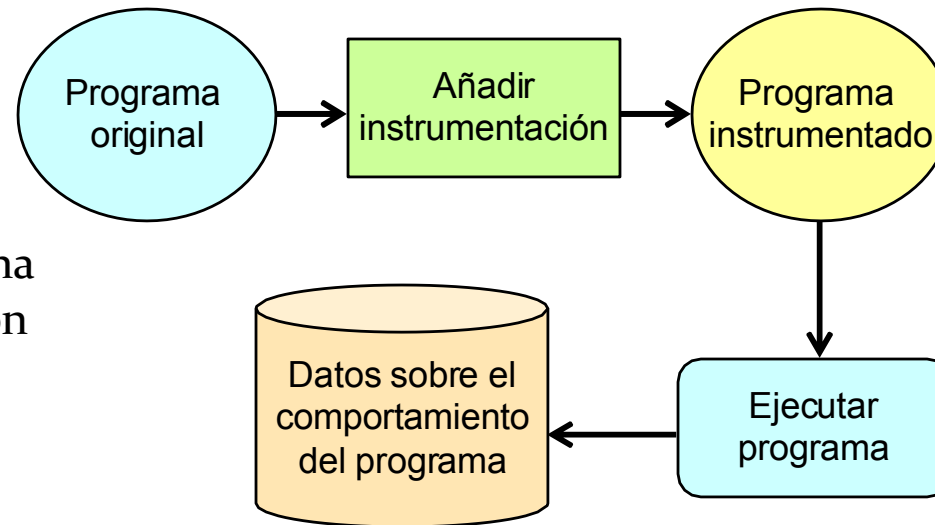
Etapas a seguir para usar un *profiler*

Compilar el programa
habilitando la recogida de
información

Ejecutar el programa
instrumentado

Ejecución más lenta porque se ha
de recoger y dejar la información
en un fichero (*profile data*)

Analizar la información
contenida en el fichero de
comportamiento



time (/usr/bin/time)

Mide el tiempo de ejecución de un programa y muestra algunas estadísticas sobre su ejecución.

real: tiempo total usado por el sistema (wall-clock CPU time = tiempo de respuesta).

user: tiempo de CPU ejecutando en modo usuario (user-state CPU time).

sys: tiempo de CPU en modo supervisor (system-state CPU time) ejecutando código del núcleo.

Cambios de contexto voluntarios: al tener que esperar a una operación de E/S (el núcleo no espera a que expire su "time slice").

```
% time ./matr_mult2
real    0m4.862s
user    0m4.841s
sys     0m0.010s

% /usr/bin/time -v ./matr_mult2
User time (seconds): 4.86
System time (seconds): 0.01
Percent of CPU this job got: 99%
Elapsed (wall clock) time 0:04.90
Maximum RSS (kbytes): 48784
Major page faults: 0
Minor page faults: 3076
Voluntary context switches: 1
Involuntary context switches: 195
Swaps: 0
etc.
```

Monitor `gprof`

Da información sobre el tiempo de ejecución y número de veces que se ejecuta cada función del programa.

Utilización de `gprof`

Instrumentación en la compilación

```
gcc prog.c -o prog -pg -g
```

Ejecución del programa y recogida de información

```
./prog
```

La información recogida se deja en el fichero `gmon.out`

Visualización de la información referida a la ejecución del programa

```
gprof prog > prog.gprof
```

Utilización del monitor `gprof`

```
#include <stdio.h>
#include <time.h>
#include <math.h>
double a=3.14,b=6.34,c=-3.03;
long y;

void main()
{
    producto(); producto(); producto();
    division(); division();
    atangente();
}

producto()
{for (i=0; i<50000000; i++) c=a*b;}

division()
{for (i=0; i<30000000; i++) c=a/b;}

atangente()
{for (i=0; i<30000000; i++) c=atan(a);}
```

Pasos

Instrumentación (`-pg`) en la compilación

Ejecución del programa y recogida de información

Obtención de la información referida a la ejecución del programa

```
% gcc bucles.c -pg -o bucles
% ./bucles
% gprof bucles > bucles.prof
```

Salida del monitor gprof

Each sample counts as 0.01 seconds

flat profile

% cumulative timeseconds	self seconds	calls	self ms/call	total ms/call	name
62.79	11.12	2	5560.00	5560.00	division
20.33	14.72	1	3600.00	3600.00	atangente
16.88	17.71	3	996.67	996.67	producto

call profile

index	% time	self	children	called	name
[1]	100.0	0.00	17.71		main [1]
		11.12	0.00	2/2	division [2]
		3.60	0.00	1/1	atangente [3]
		2.99	0.00	3/3	producto [4]
[2]	62.8	11.12	0.00	2	division [2]
[3]	20.3	3.60	0.00	1	atangente [3]
[4]	16.9	2.99	0.00	3	producto [4]

Monitor gcov

Aporta información sobre el número de veces que se ejecuta cada línea de código del programa.

Utilización de gcov

Instrumentación en la compilación

```
gcc prog.c -o prog -fprofile-arcs -ftest-coverage
```

Ejecución del programa y recogida de información

```
prog
```

La información recogida se deja en varios ficheros

Visualización de la información referida a la ejecución del programa

```
gcov prog.c (genera el fichero prog.c.gcov)
```

Opciones disponibles de gcov

Frecuencias de salto en instrucciones de salto: -b (*branch*)

Informe por funciones: -f

```
%gcc -o bucles.c -o bucles -fprofile-arcs -ftest-coverage
%gcov -f -b bucles.c
100.00% of 7 source lines executed in function main
No branches in function main
100.00% of 7 calls executed in function main
100.00% of 3 source lines executed in function producte
100.00% of 3 branches executed in function producte
  33.33% of 3 branches taken at least once in function producte
100.00% of 1 calls executed in function producte
...
100.00% of 9 branches executed in file bucles.c
  55.56% of 9 branches taken at least once in file bucles.c
100.00% of 12 calls executed in file bucles.c
Creating bucles.c.gcov.
```

V.Tune (Intel) y CodeAnalyst (AMD)

Son monitores de aplicaciones (profilers) que combinan tanto monitorización software mediante muestreo como monitorización hardware por eventos que hacen uso de contadores hardware disponibles en los últimos microprocesadores de Intel y AMD. Ambos programas funcionan tanto para Windows como para Linux, y permiten (adicionalmente a la información que nos proporciona gprof y gcov) obtener información sobre los fallos de caché, fallos de TLB, bloqueos/rupturas del cauce, fallos en la predicción de saltos, entre otros eventos, asociados a cada línea del programa.

